Repository-Specific SQL & Cypher Query Generator Handbook

This handbook documents the implemented project using local code and existing artifacts as source of truth.

Table of Contents

1. Project Definition

What it is

A local, reproducible pipeline for schema-aware query generation that supports:

- Text-to-SQL
- Text-to-Cypher
- baseline generation
- adapter fine-tuning
- multi-metric evaluation
- graph materialization in Neo4j

Why it is used

Natural-language query generation needs more than fluent text output. This project adds explicit controls for:

- schema grounding,
- parse/execution checks,
- profile-based reproducibility,
- measurable baseline-vs-finetuned comparison.

How it appears in code

- Orchestration: `src/repo_query_gen/pipeline.py`
- CLI: `scripts/run_end_to_end.py`
- Stage modules:
  - `data_prep.py`
  - `cypher.py`
  - `baselines.py`
  - `training.py`
  - `inference.py`
  - `evaluation.py`
  - `neo4j_demo.py`

Practical explanation

Each stage returns paths to generated outputs. The pipeline aggregates these paths into a run manifest. Example:

- `artifacts/manifest_tutorial_real_run.json`

2. System Workflow

End-to-end stage sequence:

1. Data preparation
2. SQL-to-Cypher label generation
3. Baseline generation
4. Fine-tuning (optional)
5. Batch inference (optional)
6. Evaluation bundle
7. Neo4j graph demo (optional)

Flow ownership in code:

- `run_end_to_end(...)` in `pipeline.py`
- per-stage runners imported from each module

3. Repository and Code Map

Top-level structure:

```
configs/
data/
docker/
notebooks/
scripts/
src/repo_query_gen/
tests/
artifacts/
docs/
```

Entry scripts:

- `scripts/prepare_data.py`
- `scripts/build_cypher_labels.py`
- `scripts/run_baselines.py`
- `scripts/train_qlora.py`
- `scripts/infer.py`
- `scripts/evaluate.py`
- `scripts/run_neo4j_demo.py`
- `scripts/run_end_to_end.py`

Runtime configuration:

- profile values: `configs/profiles.yaml`
- environment settings model: `src/repo_query_gen/config.py`
- optional overrides: `.env.example`

4. Zero-to-Hero Concepts

Each concept below includes definition, rationale, code location, and practical behavior.

4.1 Profiles and Runtime Settings

- What it is: split between `ProfileConfig` (run scale) and `Settings` (runtime knobs).
- Why: same codebase must support smoke, tutorial, and full runs.
- How in code: `config.py` (`ProfileConfig`, `Settings`, `load_profile`).
- Practical: run with `--profile fast|tutorial|full`; adjust Ollama timeout/retry settings through `Settings`.

4.2 Data Preparation

- What it is: conversion from raw rows into structured examples with schema and complexity metadata.
- Why: reliable training and evaluation require normalized, stratified, schema-aware splits.
- How in code:
  - `build_processed_dataframe(...)`
  - `_parse_create_table_blocks(...)`
  - `stratified_split(...)`
  - `_safe_text(...)` (null/NaN cleaning)
- Practical: outputs are saved under `data/processed/<profile>/` with `manifest.json`.

4.3 Deterministic SQL-to-Cypher Labeling

- What it is: rule-based translation from SQL AST to Cypher draft with quality checks.
- Why: SQL-heavy data needs aligned Cypher supervision for dual-task training.
- How in code:
  - `sql_to_cypher_deterministic(...)`
  - `validate_cypher_text(...)`
  - `build_cypher_labels_for_split(...)`
- Practical: writes `*_cypher.csv`; drops invalid SQL rows before conversion.

4.4 Schema Retrieval

- What it is: lexical overlap ranking of tables/columns for question-specific schema subsets.
- Why: reduces prompt noise and improves grounding signal.
- How in code:
  - `parse_schema_context(...)`
  - `select_schema_context(...)`
- Practical: retrieval metadata is embedded in baseline/inference outputs.

4.5 Prompt-Only Baselines

- What it is: baseline SQL/Cypher generation for Granite and Qwen via Ollama.
- Why: establishes pre-finetuning reference quality.
- How in code:
  - prompt builder `_build_prompt(...)`
  - generation `_generate(...)`
  - stage runner `run_baseline_generation(...)`
- Practical: artifacts saved in `artifacts/baseline/<profile>/` and optionally `artifacts/inference/<profile>/`.

4.6 QLoRA Fine-Tuning with Backend Resolution

- What it is: PEFT adapter tuning on quantized base model with backend selection (`hf`, `trl`, conditional `unsloth`).
- Why: improve task specialization under local hardware constraints.
- How in code:
  - backend resolver `_resolve_backend(...)`
  - LoRA attach `_attach_lora(...)`
  - runners `_run_hf_training`, `_run_trl_training`, `_run_unsloth_training`
  - stage entry `run_finetuning(...)`
- Practical: outputs include adapter weights, train/eval metrics, package versions, and GPU snapshot.

4.7 Inference and Output Normalization

- What it is: single/batch query generation with standardized postprocessing and validation.
- Why: model outputs often include preamble text; evaluation needs query-only outputs.
- How in code:
  - `postprocess_generated_query(...)`
  - `_extract_sql_statement(...)`
  - `_extract_cypher_statement(...)`
  - `validate_generated_query(...)`
- Practical: inference JSON contains query, latency, retrieval, schema refs, and validation block.

4.8 Evaluation Stack

- What it is: text metrics + syntax + schema grounding + retrieval recall + latency + optional judge + optional Spider execution.
- Why: no single metric captures query quality.
- How in code:
  - `evaluate_inference_json(...)`
  - `run_llm_judging(...)`
  - `run_spider_execution_benchmark(...)`
  - `run_evaluation_bundle(...)`
- Practical: summary JSON, per-example CSV, and plots are stored under `artifacts/evaluation/<profile>/`.

4.9 Neo4j Graph Demo

- What it is: loading query examples into graph nodes/edges plus demo analytical Cypher queries.
- Why: demonstrates graph-native inspection of source/question/query/table relationships.
- How in code:
  - `load_dataset_graph(...)`
  - `run_demo_queries(...)`
  - `run_neo4j_demo(...)`
- Practical: outputs in `artifacts/neo4j_outputs/<profile>/`.

5. Real Outputs and Metrics

All values below are read from existing repository artifacts.

5.1 Processed Data Counts

Profile	train.csv	val.csv	test.csv	train_cypher.csv	val_cypher.csv	test_cypher.csv
fast	9,000	1,200	1,200	9,000	1,200	1,200
tutorial	15,000	2,000	2,000	15,000	2,000	2,000
full	209,766	26,221	26,221	209,765	26,221	26,221

Evidence:

- `data/processed/{profile}/manifest.json`
- `data/processed/{profile}/*.csv`
- Values are artifact-backed current-state counts; they can differ from latest profile YAML defaults when artifacts were generated earlier.

5.2 Tutorial Real Run Manifest

Primary file:

- `artifacts/manifest_tutorial_real_run.json`

Generation timestamp inside manifest:

- `2026-06-20T19:58:39+00:00`

5.3 Training Metrics (Tutorial Real Run)

Run directory:

- `artifacts/training/tutorial_2026-06-20T19-14-21.185341+00-00_trl`

From `training_metadata.json`:

- requested backend: `auto`
- effective backend: `trl`
- fallback reason: `null`

- base model: `ibm-granite/granite-4.1-3b`

From `train_result.json`:

- train loss: `1.7538103103637694`
- train runtime: `87.9225`

From `eval_result.json`:

- eval loss: `1.4785621166229248`
- eval runtime: `33.5015`
- eval mean token accuracy: `0.721689356956631`

5.4 Tutorial Evaluation Summaries

Files:

- `artifacts/evaluation/tutorial/summary_baseline_granite.json`
- `artifacts/evaluation/tutorial/summary_baseline_qwen.json`
- `artifacts/evaluation/tutorial/summary_finetuned.json`

Label	exact_match	syntax_success	schema_grounding	bleu	rouge_l	
baseline_granite	0.0	1.0	0.8333333333333334	0.480422165043141	0.7554945054945055	0.63453
baseline_qwen	0.0	0.0	0.0	0.0	0.0	
finetuned	0.0	0.5	0.5	0.23091955549253448	0.334241452991453	0.286881

5.5 Tutorial Judge Aggregates

Files:

- `artifacts/evaluation/tutorial/judge_baseline_granite.json`
- `artifacts/evaluation/tutorial/judge_baseline_qwen.json`
- `artifacts/evaluation/tutorial/judge_finetuned.json`

Each file has `6` rows. Each file includes `3` timeout fallback rows (`reasoning` starts with `judge_error`).

Mean scores across all 6 rows:

Label	correctness	completeness	schema_grounding	hallucination_risk
baseline_granite	2.8333333333333335	2.5	3.0	2.5
baseline_qwen	2.8333333333333335	2.6666666666666665	3.0	2.5
finetuned	3.0	2.5	3.0	2.5

5.6 Spider Execution (Tutorial)

Files:

- `artifacts/evaluation/tutorial/spider_exec_baseline_granite.json`
- `artifacts/evaluation/tutorial/spider_exec_finetuned.json`

Both files:

- rows: `3`
- execution matches: `2`
- predicted SQL executable: `3`
- reference SQL executable: `3`

5.7 Neo4j Tutorial Outputs

File:

- `artifacts/neo4j_outputs/tutorial/graph_summary.json`

Values:

- sources: `15`
- questions: `1000`
- tables: `910`

5.8 Current Profile Completion State in This Repository

- fast: baseline/inference/evaluation/neo4j artifacts present.
- tutorial: full artifact chain present (manifest, training, inference, evaluation, spider, neo4j).
- full: processed and cypher files present; downstream baseline/inference/evaluation directories not present.

6. Execution Runbook

Environment:

```
cd /home/ahmad/AI/Repository-Specific-SQL-Cypher-Query-Generator
UV_CACHE_DIR=/tmp/uv-cache uv venv --python 3.12.10
source .venv/bin/activate
UV_CACHE_DIR=/tmp/uv-cache uv sync
```

Start Neo4j:

```
docker compose -f docker/docker-compose.neo4j.yml up -d
```

Pull models:

```
ollama pull granite4.1:3b
ollama pull qwen3.5:4b
ollama pull qwen3-embedding:4b
```

Run tutorial profile end-to-end:

```
python scripts/run_end_to_end.py \
  --profile tutorial \
  --with-inference \
  --with-judge \
  --with-spider \
  --trainer-backend auto
```

Stage-by-stage equivalent:

```
python scripts/prepare_data.py --profile tutorial
python scripts/build_cypher_labels.py --profile tutorial
python scripts/run_baselines.py --profile tutorial
python scripts/train_qlora.py --profile tutorial --backend auto
python scripts/evaluate.py --profile tutorial --with-judge --with-spider
python scripts/run_neo4j_demo.py --profile tutorial
```

7. Artifact Inspection Guide

Useful checks:

```
# summaries
cat artifacts/evaluation/tutorial/summary_baseline_granite.json
cat artifacts/evaluation/tutorial/summary_baseline_qwen.json
cat artifacts/evaluation/tutorial/summary_finetuned.json

# training metadata
cat artifacts/training/tutorial_2026-06-20T19-14-21.185341+00-00_trl/training_metadata.json

# spider execution results
cat artifacts/evaluation/tutorial/spider_exec_baseline_granite.json
cat artifacts/evaluation/tutorial/spider_exec_finetuned.json

# neo4j graph counts
cat artifacts/neo4j_outputs/tutorial/graph_summary.json
```

8. Tutorial Chapters

- 00 - Orientation and Workflow
- 01 - Configuration and Profiles
- 02 - Data Preparation Pipeline
- 03 - SQL-to-Cypher Labeling
- 04 - Schema Retrieval and Baselines
- 05 - QLoRA Fine-Tuning
- 06 - Inference Pipeline
- 07 - Evaluation and Judging
- 08 - Neo4j Graph Demo
- 09 - Real Runs and Artifact Reading
- 10 - Testing, Reliability, and Troubleshooting

9. Operational Notes and Limitations

1. Judge output interpretation

- Judge files include timeout fallback entries; always inspect `reasoning` before treating aggregate scores as stable quality signals.

2. Full profile interpretation

- Full profile currently has completed processed and cypher outputs in `data/processed/full/`.
- Downstream full-profile artifacts are not present in `artifacts/baseline/full`, `artifacts/inference/full`, `artifacts/evaluation/full`.

3. Reproducibility and evidence

- Use manifests and JSON artifacts as primary evidence.
- Avoid drawing conclusions from partially created run directories that lack `train_result.json`, `eval_result.json`, or `training_metadata.json`.